

Socket Programming with Python

A Multi-User Chat Application with Identity-Based Encryption

CS440V — Spring 2026

Ho Nguyen Kim Long · Le Chi Tam · Trieu Minh Thuan · Le Nguyen Quoc Anh

Supervisors: Truong Huu Tram, PhD & Le Quoc Huy, PhD



Table of Contents

01

Introduction

Objectives & scope of the project

02

Project Overview

Features, technologies, and architecture

03

System Design

Architecture, workflow, concurrency

04

Implementation

File structure, API workflow, flow

05

Challenges

Socket, threading, encryption issues

06

Future Improvements

ABE, multi-server, decentralization

07

Conclusion

Key achievements and takeaways

Introduction

Objective

Design and implement a socket-based application enabling secure communication in a client-server chatroom environment, integrating Identity-Based Encryption (IBE) to protect messages and authenticate participants.



Socket-Based

Reliable TCP/IP connections between multiple users sharing a chatroom



IBE Security

Identity-based encryption binds encryption keys to user identities



Multi-User

Server manages concurrent connections and message distribution

Project Overview

Main Features



Identity-Based Encryption

Secure & authenticated messaging via IBE keys



Real-Time Messaging

Instant send/receive for responsive experience



One-to-One Chat

Direct user-to-user private communication



Browser-Based GUI

User-friendly interface accessible via browser

Technologies Used

Python — Server + Client logic

Flask — RESTful API (client backend)

Socket (TCP) — Network communication

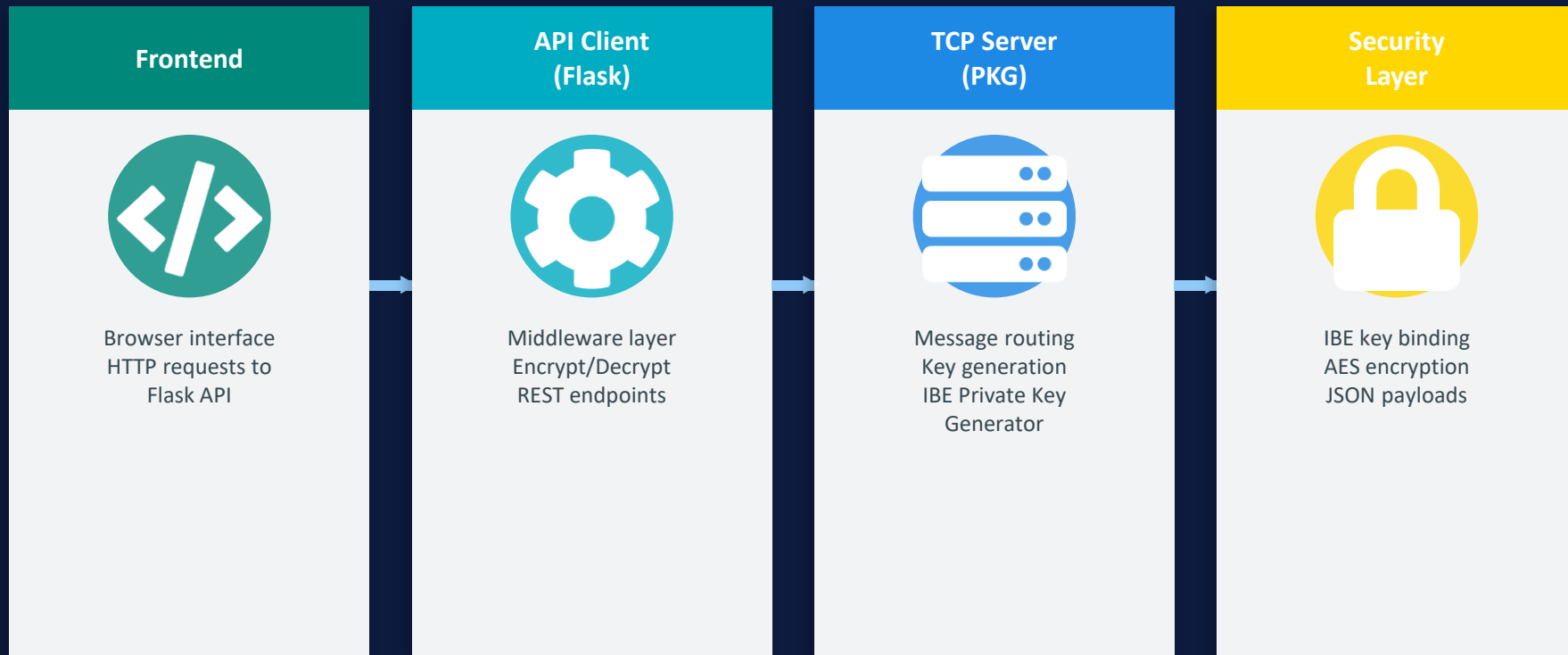
Docker — Containerized deployment

Overall Architecture

- ▶ Server: Handles connections via multi-threading
- ▶ Client Backend: Encryption + message relay
- ▶ Client Frontend: Browser-based, uses REST APIs

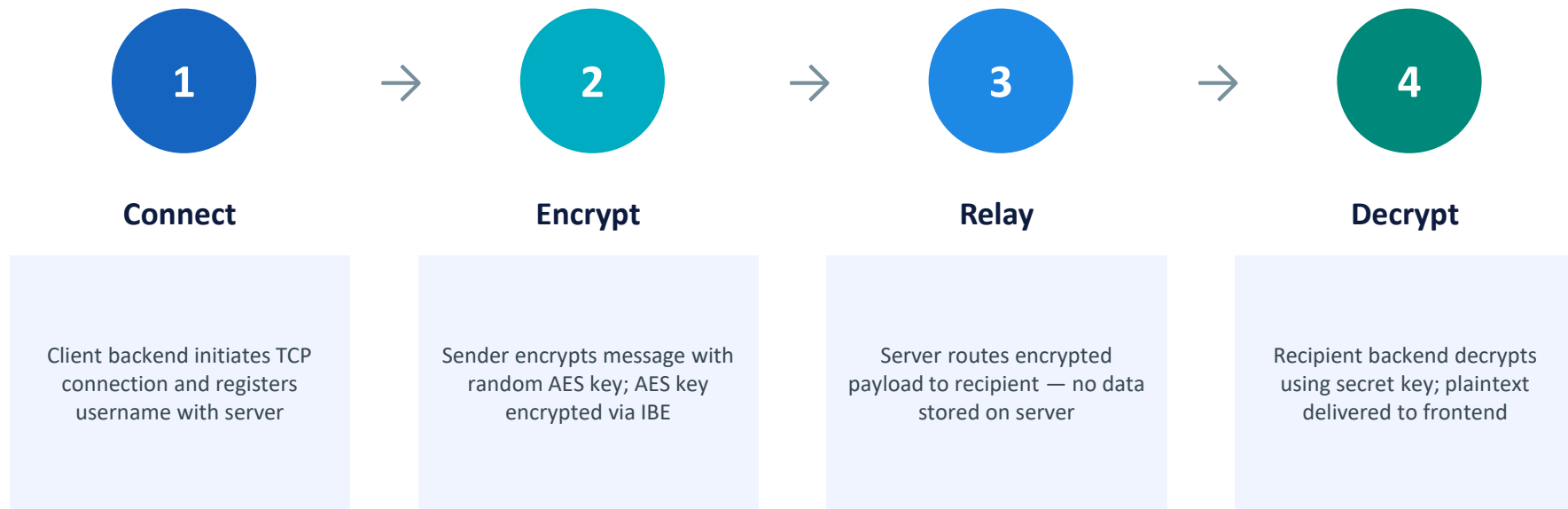
System Design

Architecture & Components



TCP/IP with JSON serialization · AES for content · IBE for session keys

System Design — Workflow & Concurrency



⚙️ Concurrency Handling

Multi-threading: Each client connection handled by a dedicated thread | **Data Transmission:** JSON serialized messages over TCP · IBE for session keys · AES for content

Implementation — File Structure

ClientContainer

<code>API_client/client_api.py</code>	Socket + crypto logic
<code>API_client/requirements.txt</code>	Python dependencies
<code>frontend/src/</code>	React/JS frontend source
<code>frontend/index.html</code>	Web interface entry point
<code>frontend/vite.config.js</code>	Vite build configuration
<code>Dockerfile</code>	Container build & deploy

ServerContainer

<code>API_server/server_api.py</code>	Socket handler + key mgmt
<code>API_server/requirements.txt</code>	Python dependencies
<code>API_server/Dockerfile</code>	Testing Dockerfile
<code>Dockerfile</code>	Main server container
<code>README.md</code>	Server documentation

Implementation — API Workflow

API Server (TCP Socket)

register

Register client, generate IBE keys (MPK, SK), return online users

message

Relay encrypted payload to recipient (no storage)

request_users

Broadcast updated list of online users to all clients

API Client (Flask RESTful)

POST /connect

Establish TCP connection, register user, start listener thread

POST /send

Encrypt message + AES key via IBE, send to server

GET /messages

Retrieve and clear buffered decrypted messages

GET /users

Return real-time online user list

Challenges

Socket Connection Issues

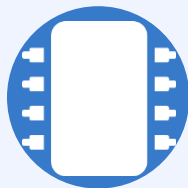


⚠ Unexpected client disconnections — server state becomes inconsistent

⚠ Client may believe it's connected after server dropped the link

⚠ Users miss messages or appear online when actually disconnected

Multi-Threading Challenges



⚠ Hardware limits — high CPU/memory usage with many concurrent clients

⚠ Race conditions, deadlocks, resource exhaustion risks

⚠ Requires careful thread sync and resource allocation optimization

IBE Centralization Risk



⚠ Server acts as sole Private Key Generator (PKG) — single point of failure

⚠ Attacker targeting server can access all cryptographic material

⚠ Concentrates security responsibility; requires strong server-side safeguards

Future Improvements

Attribute-Based Encryption (ABE)



- ✓ Replace one-to-one IBE with ABE for group chat
- ✓ Encrypt based on attributes like group membership
- ✓ Multiple users decrypt if they satisfy attribute policy

Multi-Server Architecture



- ✓ Deploy multiple servers to distribute workload
- ✓ Reduce single-server bottleneck for high traffic
- ✓ Improved scalability and fault tolerance

Reduce Centralized Security Risks



- ✓ Adopt multi-authority ABE to remove single PKG
- ✓ Decentralized key management across multiple authorities
- ✓ Reduces risk of full system compromise via one server

Conclusion

Key Achievements



Successfully implemented socket-based multi-user chat with IBE encryption



Multi-threaded server manages concurrent clients reliably



Docker containerization ensures portable, reproducible deployment



Modular design: server, client backend, and browser frontend decoupled

Lessons Learned

- ◆ Socket programming reliability
- ◆ Thread management complexity
- ◆ IBE centralization trade-offs
- ◆ Security vs. usability balance

Thank You!

Questions & Discussion

Ho Nguyen Kim Long · Le Chi Tam · Trieu Minh Thuan · Le Nguyen Quoc Anh

Supervised by: Truong Huu Tram, PhD & Le Quoc Huy, PhD

🔗 github.com/MrNhoiBong/Chatting_Encryption